

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

راهنمای جامع
حالت‌های کم‌مصرف
در میکروکنترلرهای CH32

از تفاوت SLEEP، STOP و STANDBY تا نمونه‌های عملی بیدارسازی با RTC، پایهٔ WKUP، ماژول AWU، تایمر LPTIM و پایشگر PVD

در این جزوه، ترجمهٔ فارسی سند رسمی AN01000 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را با ترجمهٔ فنی و روان و با حفظ اصطلاحات اصلی سخت‌افزار می‌خوانید؛ تفاوت حالت‌های کم‌مصرف و شیوهٔ انتخاب حالت مناسب، کاربرد منابع بیدارسازی همراه با نمونه‌های پیکربندی، روش‌های بهینه‌سازی جریان مصرفی در حالت خواب از سه منظر نرم‌افزار، سخت‌افزار و خطای اندازه‌گیری، و نکات مهم حین خواب و بیدارسازی را گام‌به‌گام مرور می‌کنیم.

حالت‌های کم‌مصرف · بیدارسازی با RTC · پایهٔ WKUP · ماژول AWU · تایمر LPTIM
پایشگر PVD · بهینه‌سازی جریان خواب · واژه‌نامهٔ فنی

ترجمهٔ فارسی AN01000 V1.0 · نسخهٔ ۱٫۰ — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | حالت‌های کم‌مصرف ۳

1.1.1 حالت SLEEP ۴
1.1.2 حالت STOP ۴
1.1.3 حالت STANDBY ۴
1.2.1 بیدارسازی با هشدار RTC ۴
1.2.2 بیدارسازی با پایهٔ WKUP ۶
1.2.3 بیدارسازی خودکار AWU ۷
1.2.4 بیدارسازی با LPTIM ۸
1.2.5 بیدارسازی با PVD ۹

۳ بخش ۲ | نکات و ملاحظات کم‌مصرف ۹

2.1.1 پیکربندی پایه‌های MCU ۹
2.1.2 خاموش‌کردن جانبی‌ها و کاهش فرکانس ۱۰
2.1.3 صرفه‌جویی ریگولاتور و RAM ۱۰
2.1.4 اجرای کد از SRAM ۱۰
2.1.5 بررسی مدارهای اطراف ۱۱
2.1.6 کاهش خطای اندازه‌گیری جریان ۱۱
2.2.1 خواب و بیدارسازی صحیح ۱۲
2.2.2 حفظ سطح پایه‌ها در STANDBY ۱۲
2.2.3 دریافت منبع رویداد WFE ۱۲
2.2.4 حفظ RAM در STANDBY ۱۳

۴ تاریخچهٔ نسخه‌ها و بیانیه ۱۳

۵ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۳

فهرست شکل‌ها و جدول‌ها

شکل ۱-۱ نمونهٔ پیکربندی بیدارسازی با هشدار RTC ۵
شکل ۱-۲ نمونهٔ وقفهٔ بیدارسازی با هشدار RTC ۶
شکل ۱-۳ نمونهٔ بیدارسازی با پایهٔ WKUP ۷
شکل ۱-۴ نمونهٔ بیدارسازی خودکار AWU ۸
شکل ۲-۱ نمودار اندازه‌گیری جریان ۱۱

جدول ۱-۱ مرور کلی حالت‌های کم‌مصرف ۳
جدول ۱ دامنهٔ کاربرد سند ۳
جدول ۲ تاریخچهٔ نسخه‌ها ۱۳
جدول ۳ واژگان کلیدی ۱۳

چکیده — میکروکنترلرهای سری CH32 حالت‌های کم‌مصرف متعددی ارائه می‌کنند. این سند کاربردی تفاوت حالت‌های کم‌مصرف مختلف را فهرست می‌کند، نمونه‌هایی از بیدارسازی سیستم از حالت کم‌مصرف به‌وسیلهٔ بخشی از منابع بیدارسازی ارائه می‌دهد، روش‌های بهینه‌سازی جریان مصرفی در حالت کم‌مصرف را از منظر پیکربندی نرم‌افزار، طراحی سخت‌افزار و خطای اندازه‌گیری معرفی می‌کند و نکات موردتوجه و ترفندهای کاربردی در فرایند خواب و بیدارسازی را شرح می‌دهد.

مقدمه

توضیحات و دامنهٔ کاربرد

میکروکنترلرهای سری CH32 چندین حالت کم‌مصرف ارائه می‌کنند. این سند کاربردی تفاوت حالت‌های کم‌مصرف مختلف را فهرست کرده و نمونه‌هایی از کاربرد برخی منابع بیدارسازی برای بیدارکردن سیستم از حالت کم‌مصرف ارائه می‌دهد؛ روش‌های بهینه‌سازی جریان مصرفی در حالت کم‌مصرف را از منظرهای پیکربندی نرم‌افزار، طراحی سخت‌افزار و خطای اندازه‌گیری معرفی می‌کند و نکات موردتوجه و ترفندهای کاربردی در فرایند خواب و بیدارسازی را شرح می‌دهد.

جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
MCU عمومی	میکروکنترلرهای عمومی CH32

بخش ۱

حالت‌های کم‌مصرف

۱.۱ معرفی حالت‌های کم‌مصرف

میکروکنترلرهای سری CH32 حالت‌های کم‌مصرف متعددی مانند SLEEP، STOP و STANDBY ارائه می‌کنند و روش‌های بیدارسازی متفاوتی برای خروج سیستم از این حالت‌ها در اختیار می‌گذارند. هنگامی‌که سیستم نیازی به ادامهٔ اجرا ندارد، می‌توان متناسب با نیاز سیستم، حالت کم‌مصرف مناسب را برای صرفه‌جویی در مصرف توان انتخاب کرد.

جدول ۱-۱ مرور کلی حالت‌های کم‌مصرف

حالت	نحوهٔ ورود	بیدارسازی	توضیح
SLEEP	WFI	بیدارسازی با هر وقفه	ساعت هسته خاموش می‌شود و از سایر ساعت‌ها اثری نیست؛ همهٔ پایه‌های I/O حفظ می‌شوند و پس از بیدارسازی اجرا ادامه می‌یابد.
	WFE	بیدارسازی با هر رویداد	
STOP (۱)	قرار دادن SLEEPDEEP روی ۱ و PDDS روی ۰	WFI یا WFE؛ هر وقفه یا رویداد خارجی (۱)	ساعت‌های HSE، HSI، PLL و ساعت جانبی‌ها خاموش می‌شوند؛ همهٔ پایه‌های I/O حفظ می‌شوند و پس از بیدارسازی اجرا ادامه می‌یابد.
STANDBY (۱)	قرار دادن SLEEPDEEP روی ۱ و PDDS روی ۱	WFI یا WFE؛ هر رویداد و موارد مشابه (۱)	ساعت‌های HSE، HSI، PLL و ساعت جانبی‌ها خاموش می‌شوند؛ پس از بیدارسازی اجرا ادامه می‌یابد یا سیستم reset می‌شود.

توجه: برخی محصولات حالت STOP یا STANDBY را ارائه نمی‌کنند؛ منابع بیدارسازی در محصولات مختلف متفاوت است و در برخی محصولات، پس از بیدارسازی از حالت STANDBY سیستم به اجرا ادامه می‌دهد. برای جزئیات دقیق‌تر به دیتاشیت همان تراشه مراجعه کنید.

◆ ۱.۱.۱ حالت SLEEP

با اجرای دستور `WFI` یا `WFE` وارد حالت خواب می‌شوید. در حالت SLEEP، همهٔ پایه‌های I/O وضعیت خود را در حالت اجرا حفظ می‌کنند و ساعت همهٔ جانبی‌ها نیز به‌صورت عادی فعال است؛ بنابراین پیش از ورود به خواب، تا جای ممکن ساعت جانبی‌های غیرموردنیاز را خاموش کنید تا مصرف توان کاهش یابد. زمان لازم برای بیدارسازی در این حالت از همهٔ حالت‌ها کمتر است.

◆ ۱.۱.۲ حالت STOP

در این حالت دامنهٔ ساعت‌های فرکانس‌بالا (HSE/HSI/PLL) خاموش می‌شود، محتوای SRAM و رجیسترها حفظ می‌گردد و وضعیت پایه‌های I/O نیز حفظ می‌شود. پس از بیدارسازی از این حالت، سیستم می‌تواند به اجرا ادامه دهد و HSI ساعت پیش‌فرض سیستم است. اگر در لحظهٔ ورود به این حالت برنامه‌نویسی Flash در جریان باشد، سیستم تا تکمیل دسترسی به حافظه وارد حالت توقف نمی‌شود؛ همچنین اگر دسترسی به PB در جریان باشد، تا پایان آن دسترسی وارد حالت توقف نمی‌شود. حالت STOP تعادلی میان مصرف توان و انعطاف‌پذیری برقرار می‌کند و برای بیشتر کاربردهای کم‌مصرف مناسب است.

◆ ۱.۱.۳ حالت STANDBY

در حالت آماده‌به‌کار، ریگولاتور ولتاژ خاموش می‌شود و همهٔ مدارها به‌جز مدار بیدارسازی و مدارهای دامنهٔ پشتیبان (Backup) از برق خارج می‌شوند تا کمترین مصرف توان حاصل شود. پس از خروج در شرایط بیدارسازی مشخص‌شده، میکروکنترلر reset می‌شود و reset از نوع تغذیه (Power Reset) اجرا می‌گردد (در برخی تراشه‌ها مانند سری‌های CH32V00x و CH32X03x، در حالت STANDBY محتوای SRAM و رجیسترها حفظ می‌شود، وضعیت پایه‌های I/O حفظ می‌گردد، پس از بیدارسازی سیستم به اجرا ادامه می‌دهد و HSI ساعت پیش‌فرض سیستم است). حالت STANDBY کمترین مصرف توان را دارد، اما معمولاً پس از بیدارسازی باید سیستم دوباره راه‌اندازی شود و برای سناریوهایی که الزام واقع‌زمانی ندارند مناسب است.

◆ ۱.۲ کاربرد برخی منابع بیدارسازی

◆ ۱.۲.۱ بیدارسازی با هشدار RTC

RTC می‌تواند بدون نیاز به وقفهٔ خارجی، بیدارسازی خودکار را ممکن کند. با برنامه‌ریزی روی مبنای زمانی، می‌توان به‌صورت دوره‌ای از حالت‌های STOP یا STANDBY بیدار شد.

برای مثال در CH32F/V20x_V30x می‌توان کریستال خارجی کم‌فرکانس دقیق 32.768kHz یعنی LSE را به‌عنوان منبع ساعت RTC انتخاب کرد یا نوسانگر داخلی LSI را به‌عنوان منبع ساعت RTC به کار برد؛ شاخص دقت و مصرف توان LSI نسبت به LSE ضعیف‌تر است. اگر از بیدارسازی رویدادی استفاده می‌کنید، لازم است سیگنال درخواست رویدادِ کانال ۱۷ وقفهٔ

خارجی فعال شود؛ اگر از بیدارسازی وقفه‌ای استفاده می‌کنید، باید سیگنال درخواست وقفهٔ کانال ۱۷ وقفهٔ خارجی فعال شود، وقفهٔ خارجی هشدار فعال گردد و تابع سرویس وقفه نوشته شود. حالت STOP هم از بیدارسازی با رویداد و هم با وقفه پشتیبانی می‌کند و پس از بیدارسازی سیستم به اجرا ادامه می‌دهد؛ اما حالت STANDBY فقط با رویداد قابل بیدارسازی است، نیازی به پیکربندی خط ۱۷ وقفهٔ خارجی ندارد و پس از بیدارسازی سیستم reset می‌شود.

```
void RTC_Alarm_WakeUp( u32 SetAlarm )
{
    EXTI_InitTypeDef EXTI_InitStructure = {0};

    /* Enable PWR and BKP clocks */
    RCC_PB1PeriphClockCmd(RCC_PB1Periph_PWR | RCC_PB1Periph_BKP, ENABLE);
    PWR_BackupAccessCmd(ENABLE);
    RTC_ClearITPendingBit(RTC_IT_ALR);
    RTC_ClearITPendingBit(RTC_IT_SEC);
    /* Enable LSE oscillator */
    RCC_LSEConfig(RCC_LSE_ON);
    while(RCC_GetFlagStatus(RCC_FLAG_LSERDY) != SET);
    RCC_RTCCLKConfig(RCC_RTCCLKSource_LSE);
    /* Enable the RTC clock */
    RCC_RTCCLKCmd(ENABLE);
    RTC_WaitForLastTask();
    RTC_WaitForSynchro();
    /* Enable alarm interrupt */
    RTC_ITConfig( RTC_IT_ALR, ENABLE );
    RTC_ITConfig( RTC_IT_OW, ENABLE );
    RTC_WaitForLastTask();
    RTC_EnterConfigMode();
    /* Set the RTC prescaler value */
    RTC_SetPrescaler(32767);
    RTC_WaitForLastTask();
    RTC_SetCounter(0);
    RTC_WaitForLastTask();
    RTC_ExitConfigMode();

#if(Enter_MODE == Enter_WFI)
    /* Enable the RTC alarm interrupt */
     NVIC_SetPriority(RTCAlarm_IRQn, 1);
     NVIC_EnableIRQ(RTCAlarm_IRQn);
#endif
    /* Configure EXTI line */
    EXTI_InitStructure.EXTI_Line = EXTI_Line17;
#if(Enter_MODE == Enter_WFI)
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
#elif(Enter_MODE == Enter_WFE)
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Event;
#endif
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    /* Set the RTC alarm value */
    RTC_SetAlarm(RTC_GetCounter()+10);
    RTC_WaitForLastTask();
    /* Enters STOP mode */
#if(Enter_MODE == Enter_WFI)
    /* Match with wake-up by interrupt */
    PWR_EnterSTOPMode(PWR_Regulator_LowPower,PWR_STOPEntry_WFI);
#elif(Enter_MODE == Enter_WFE)
    /* Match with wake-up by event */
    PWR_EnterSTOPMode(PWR_Regulator_LowPower,PWR_STOPEntry_WFE);
#endif
}
```

شکل ۱-۱ نمونهٔ پیکربندی بیدارسازی با هشدار RTC

ساعت جانبی‌های PWR و BKP فعال می‌شود و منبع ساعت RTC پیکربندی می‌گردد؛ پس از پایدارشدن ساعت LSE/LSI، ساعت RTC و وقفهٔ هشدار فعال می‌شوند، ضریب تقسیم ساعت RTC و مقدار هشدار تنظیم می‌شود و حالت STOP اجرا می‌گردد. پس از ۱۰ ثانیه وقفهٔ هشدار رخ می‌دهد و سیستم بیدار می‌شود. باید توجه داشت که پس از بیداری، HSI ساعت پیش‌فرض سیستم است؛ برای تضمین اجرای صحیح برنامه توصیه می‌شود مقداردهی اولیهٔ ساعت سیستم دوباره انجام شود.

```
void RTCAlarm_IRQHandler(void)
{
    /* After waking up, HSI is the default system clock.To ensure the normal operation of
the system, it is recommended to perform clock initialization */
    SystemInit();
   if(EXTI_GetITStatus(EXTI_Line17)!=RESET)
   {
        /* Clears EXTI line and alarm clock flag */
        EXTI_ClearITPendingBit(EXTI_Line17);
        RTC_ClearITPendingBit(RTC_IT_ALR);
        RTC_WaitForLastTask();
   }
}
```

شکل ۱-۲ نمونهٔ وقفهٔ بیدارسازی با هشدار RTC

در نمونهٔ بالا از تابع وقفهٔ هشدار RTC یعنی `RTCAlarm_IRQHandler()` استفاده شده و از وقفهٔ سراسری RTC یعنی `RTC_IRQHandler()` استفاده نشده است. اگر قرار باشد هر دو تابع وقفه هم‌زمان به کار روند، باید چنین تنظیمی انجام دهید که حفره‌ای باقی نماند: اولویت تابع `RTCAlarm_IRQHandler()` حتماً باید بالاتر از `RTC_IRQHandler()` باشد. دلیل آن این است:

اندکی پیش از وقوع وقفهٔ هشدار، حتماً وقفهٔ ثانیه رخ داده است؛ پس ابتدا تابع وقفهٔ `RTC_IRQHandler()` اجرا می‌شود و در جریان اجرای آن، پرچم وقفهٔ هشدار دوباره در وضعیت معلق قرار می‌گیرد. از آنجا که `RTC_IRQHandler()` تابع وقفهٔ سراسری است، برنامه تنها زمانی از این تابع خارج می‌شود که همهٔ پرچم‌های وقفه پاک شده باشند. اگر `RTC_IRQHandler()` و `RTCAlarm_IRQHandler()` اولویت یکسانی داشته باشند، برای خارج‌شدن برنامه از `RTC_IRQHandler()` ناگزیر باید پرچم وقفهٔ هشدار را پاک کنید (اگر پرچم را پاک نکنید، برنامه در `RTC_IRQHandler()` گیر می‌کند). اما پس از پاک‌کردن پرچم وقفهٔ هشدار، برنامه دیگر وارد `RTCAlarm_IRQHandler()` نمی‌شود و در نتیجه تابع `RTCAlarm_IRQHandler()` هرگز اجرا نخواهد شد.

با تنظیم اولویت تابع وقفهٔ هشدار `RTCAlarm_IRQHandler()` بالاتر از تابع وقفهٔ سراسری `RTC_IRQHandler()`، هنگام اجرای تابع سراسری اگر وقفهٔ هشدار رخ دهد، وقفه به‌صورت تودرتو برای اجرای `RTCAlarm_IRQHandler()` پرش می‌کند و پس از اتمام آن، اجرای `RTC_IRQHandler()` ادامه می‌یابد.

◆ ۱.۲.۲ بیدارسازی با پایهٔ WKUP

سری‌هایی مانند CH32F/V10x_20x_V30x و CH32L103 دارای رابط سخت‌افزاری اختصاصی برای بیدارسازی MCU از حالت کم‌مصرف هستند: پایهٔ WKUP (PA0). پس از فعال‌سازی، پایهٔ WKUP به‌صورت اجباری در حالت ورودی با مقاومت کشیدن به پایین (Pull-down) پیکربندی می‌شود و لبهٔ بالارونده WKUP برای بیدارکردن MCU از حالت STANDBY به کار می‌رود.

توصیه می‌شود در مقداردهی اولیه، پرچم بیدارسازی (`PWR_GetFlagStatus(PWR_FLAG_WU)`) بررسی شود تا مشخص گردد آیا راه‌اندازی ناشی از بیدارسازی بوده است یا خیر.

```
void WakeUp_Config(void)
{
    RCC_PB1PeriphClockCmd(RCC_PB1Periph_PWR, ENABLE);

    if(PWR_GetFlagStatus(PWR_FLAG_WU) == SET)
    {
        printf("wake up.. \r\n");
    }
    else
    {
        printf("start standby... \r\n");

        /* Enable the WKUP Pin functionality */
        PWR_WakeUpPinCmd(ENABLE);
        /* Enters STANDBY mode */
        PWR_EnterSTANDBYMode();
    }
}
```

شکل ۱-۳ نمونهٔ بیدارسازی با پایهٔ WKUP

◆ ۱.۲.۳ بیدارسازی خودکار AWU

برای MCUهای سری RISC-V که فاقد قابلیت RTC هستند – مانند سری‌های CH32V00x و CH32X03x – هنگامی‌که در حالت کم‌مصرف نیاز به بیدارسازی خودکار باشد، قابلیت AWU ارائه شده است. AWU از بیدارسازی MCU به دو شیوهٔ وقفه یا رویداد پشتیبانی می‌کند؛ در بیدارسازی به شیوهٔ رویداد، کد ساده‌تر است و فضای کمتری از Flash می‌گیرد و برای MCUهای ظرفیت‌کم انتخاب بهتری به شمار می‌رود.

ماژول AWU یک شمارندهٔ ۶ بیتی خودافزا است؛ هنگامی‌که مقدار شمارش با مقدار نوشته‌شده در آن برابر شود، از حالت STOP یا STANDBY بیدار می‌شود. فرمول محاسبهٔ زمان AWU به این صورت است:

$$T = \text{Windows Value} \times (\text{Prescaler} / \text{AWU_CLK}) \quad (s)$$

برای مثال در CH32V00x با ضریب تقسیم ۱۰۲۴۰ و مقدار پنجرهٔ ۲۵: نوسانگر داخلی کم‌فرکانس 128kHz یعنی LSI مبنای زمانی شمارش بیدارسازی خودکار است و پس از تقسیم بر ۱۰۲۴۰، زمان هر شمارش 1/12.5 ثانیه می‌شود؛ در نتیجه فاصلهٔ زمانی بیدارسازی برابر ۲۵ ÷ ۱۲٫۵ = ۲ ثانیه است. روش محاسبه در CH32X035 مانند CH32V00x است، اما منبع ساعت ماژول AWU متفاوت است: منبع ساعت ماژول AWU در CH32X035، ساعت تقسیم‌شدهٔ 47KHz از ساعت سریع داخلی HSI است.

روش پیکربندی بیدارسازی رویدادی AWU: ابتدا خط وقفهٔ خارجی متناظر با AWU در حالت رویداد پیکربندی می‌شود، ساعت PWR فعال می‌گردد و LSI روشن می‌شود؛ پس از پایدارشدن LSI، مقدار پیش‌تقسیم‌کننده و مقدار مقایسهٔ پنجرهٔ AWU تنظیم می‌شود و پس از تکمیل پیکربندی، قابلیت AWU فعال می‌گردد و با فرمان `WFE` ، MCU وارد حالت کم‌مصرف

می‌شود. به این ترتیب هدف بیدارسازی MCU در حالت کم‌مصرف از طریق رویداد محقق می‌شود. ضمناً AWU فقط یک بار نیاز به پیکربندی دارد و نیازی به پیکربندی مجدد پس از هر بیدارسازی نیست.

روش پیکربندی بیدارسازی وقفه‌ای AWU: ابتدا خط وقفهٔ خارجی متناظر با AWU در حالت وقفه پیکربندی می‌شود (در CH32V00x چون AWU به خط ۹ وقفهٔ خارجی متصل است و وقفه‌های خارجی فقط ۰ تا ۷ هستند، لازم است وقفهٔ AWU فعال شود و در تابع سرویس وقفه، پرچم وقفهٔ خط ۹ وقفهٔ خارجی پاک گردد؛ در CH32X035 کافی است مستقیماً وارد وقفهٔ خارجی متناظر شوید و پرچم‌های وقفهٔ مربوطه را پاک کنید). سپس وقفهٔ مربوطه فعال و اولویت وقفه تنظیم می‌شود، ساعت جانبی PWR فعال می‌گردد، پیش‌تقسیم‌کنندهٔ AWU و مقدار مقایسهٔ پنجرهٔ AWU پیکربندی و AWU فعال می‌شود و با اجرای `WFI` وارد حالت کم‌مصرف می‌شوید. در این حالت بیدارسازی زمان‌بندی‌شدهٔ MCU به شیوهٔ وقفه از طریق AWU محقق می‌شود.

```c
void AWU_WakeUp(void)
{
    EXTI_InitTypeDef EXTI_InitStructure = {0};
    RCC_PB2PeriphClockCmd(RCC_PB2Periph_AFIO, ENABLE);

    EXTI_InitStructure.EXTI_Line = EXTI_Line9;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Event;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    RCC_LSICmd(ENABLE);
    while(RCC_GetFlagStatus(RCC_FLAG_LSIRDY) == RESET);
    /* Set the Auto Wake up Prescaler */
    PWR_AWU_SetPrescaler(PWR_AWU_Prescaler_10240);
    /* Set the SWU window value */
    PWR_AWU_SetWindowValue(25);
    /* Enable the Auto WakeUp functionality */
    PWR_AutoWakeUpCmd(ENABLE);
    /* Enter STANDBY mode */
    PWR_EnterSTANDBYMode(PWR_STANDBYEntry_WFE);
    USART_Printf_Init(115200);
    printf("Auto wake up!!!!!\r\n");
}
```

شکل ۱-۴ نمونهٔ بیدارسازی خودکار AWU

◆ ۱.۲.۴ بیدارسازی با LPTIM

جانبی LPTIM در سری CH32L103 تایمری کم‌مصرف است که مخصوصاً برای سناریوهای کاربردی کم‌مصرف طراحی شده است. LPTIM تایمری ۱۶ بیتی با شمارش صعودی است و منابع ساعت قابل‌انتخاب متنوعی دارد (منابع ساعت داخلی: LSE، LSI، HSI یا ساعت PB1؛ منبع ساعت خارجی: ساعت خارجی روی ورودی LPTIM)؛ به همین دلیل LPTIM می‌تواند در همهٔ حالت‌های تغذیه به‌جز حالت آماده‌به‌کار کار کند. LPTIM حتی بدون منبع ساعت داخلی نیز می‌تواند کار کند و بر همین اساس می‌توان از آن به‌عنوان «شمارندهٔ پالس» استفاده کرد. افزون بر این، LPTIM قادر است سیستم را از حالت کم‌مصرف بیدار کند؛ بنابراین LPTIM برای پیاده‌سازی «قابلیت زمان‌بندی و تایم‌اوت» با مصرف توان بسیار کم بسیار مناسب است.

اثر حالت‌های کم‌مصرف بر LPTIM:

- حالت SLEEP: بدون تأثیر؛ وقفهٔ LPTIM موجب خروج دستگاه از حالت خواب می‌شود.
- حالت STOP: جانبی LPTIM در زمانی که با LSE یا LSI ساعت‌دهی می‌شود فعال است؛ وقفهٔ LPTIM موجب خروج دستگاه از حالت توقف می‌شود.
- حالت STANDBY: جانبی LPTIM برق‌قطع شده است و باید پس از خروج از حالت آماده‌به‌کار دوباره مقداردهی اولیه شود.

اگر از بیدارسازی رویدادی استفاده می‌کنید، باید سیگنال درخواست رویداد کانال ۲۱ وقفهٔ خارجی فعال شود؛ اگر از بیدارسازی وقفه‌ای استفاده می‌کنید، باید سیگنال درخواست وقفهٔ کانال ۲۱ وقفهٔ خارجی فعال شود، وقفهٔ خارجی LPTIM فعال گردد و تابع سرویس وقفه نوشته شود.

۱.۲.۵ بیدارسازی با PVD

پایشگر ولتاژ قابل‌برنامه‌ریزی PVD عمدتاً برای پایش تغییرات منبع تغذیهٔ اصلی سیستم به کار می‌رود؛ ولتاژ با آستانهٔ تنظیم‌شده توسط بیت‌های PLS[2:0] در رجیستر کنترل توان PWR_CTLR مقایسه می‌شود و با هماهنگی پیکربندی رجیسترهای وقفهٔ خارجی (EXTI) می‌تواند رویداد یا وقفهٔ مربوطه را تولید کند و سیستم را از حالت SLEEP یا STOP بیدار سازد.

روش پیکربندی به این صورت است: ابتدا ساعت PWR فعال می‌شود، سپس آستانهٔ پایش ولتاژ تنظیم می‌گردد و پس از آن بیت PVDE برای روشن‌کردن قابلیت پایش ولتاژ تغذیه فعال می‌شود. قابلیت PVD به‌صورت داخلی به خط ۱۶ ماژول EXTI متصل است و تنظیم تحریف لبهٔ بالارونده/پایین‌رونده روی آن اعمال می‌شود؛ با پیکربندی رجیسترهای مربوطهٔ EXTI، هرگاه VDD به زیر آستانهٔ PVD سقوط کند یا از آن بالاتر برود، رویداد یا وقفهٔ PVD تولید می‌شود. حالت STANDBY از بیدارسازی با PVD پشتیبانی نمی‌کند.

بخش ۲

نکات و ملاحظات کم‌مصرف

۲.۱ چگونه جریان خواب تراشه را بهینه کنیم؟

۲.۱.۱ پیکربندی پایه‌های MCU

- پیش از ورود به حالت کم‌مصرف، لازم است همهٔ پایه‌های I/O به‌صورت ورودی Pull-up/Pull-down یا ورودی آنالوگ پیکربندی شوند تا جریان نشتی ناشی از شناورماندن (Floating) پایه‌های I/O تراشه بروز نکند. دقت کنید که آیا پایه‌های تراشه از ورودی Pull-up/Pull-down یا ورودی آنالوگ پشتیبانی می‌کنند یا خیر (در غیر این صورت پیکربندی اثر نمی‌کند و حالت پیش‌فرض شناور باقی می‌ماند) و متناسب با شرایط انتخاب کنید.
- در مدل‌های با بستهٔ کوچک، برای پایه‌هایی که در مقایسه با بزرگ‌ترین بستهٔ آن خانواده، به بستهٔ کوچک‌تر نیاورده شده‌اند، توصیه می‌شود آن‌ها را نیز به‌صورت ورودی Pull-up/Pull-down یا آنالوگ پیکربندی کنید؛ در غیر این صورت ممکن است بر شاخص جریان اثر بگذارد.

- رابط اشکال‌زدایی SWD را به‌صورت کارکرد GPIO آزاد کنید و به‌صورت ورودی Pull-up/Pull-down یا آنالوگ پیکربندی نمایید (پس از بیدارسازی قابلیت SWD بازیابی می‌شود): `GPIO_PinRemapConfig(GPIO_Remap_SWJ_Disable, ENABLE)`.

۲.۱.۲ خاموش‌کردن جانبی‌ها و کاهش فرکانس اصلی

- در حالت SLEEP هسته متوقف می‌شود، اما همهٔ جانبی‌ها (از جمله جانبی‌های اختصاصی هسته) به کار خود ادامه می‌دهند؛ هرچه فرکانس ساعت سیستم بالاتر و تعداد جانبی‌های فعال بیشتر باشد، مصرف توان بیشتر است. توصیه می‌شود پیش از خواب، جانبی‌های استفاده‌نشده را غیرفعال کنید، PLL را خاموش و به ساعت کم‌سرعت سوئیچ کنید تا صرفه‌جویی حاصل شود. ضمناً در برخی محصولات می‌توان HSI را در حالت کم‌مصرف پیکربندی کرد؛ برای مثال در سری CH32L103 پس از فعال‌شدن حالت کم‌مصرف HSI، فرکانس خروجی از 8MHz به 1MHz کاهش می‌یابد (هنگام استفاده از UART با HSI برابر 1MHz، نرخ باود تا زیر 62.5Kbps پشتیبانی می‌شود).
- از خاموش‌سازی ناقص جانبی‌ها بپرهیزید. در حالت STOP یا در حالت STANDBY برخی تراشه‌ها، ریگولاتور ولتاژ خاموش نمی‌شود؛ برای مثال اگر ADC پیش از خواب فعال شده باشد، پس از خواب ساعت جانبی ADC به‌طور خودکار بسته می‌شود، اما بخشی از مدار داخلی ADC همچنان مقداری جریان مصرف می‌کند؛ بنابراین پیش از خواب باید جانبی‌ها را غیرفعال کنید.

۲.۱.۳ فعال‌سازی حالت صرفه‌جویی ریگولاتور ولتاژ و RAM

برای مثال در CH32F/V20x_V30x:

در حالت STOP می‌توان بیت LPDS را انتخاب کرد: با LPDS=0 ریگولاتور ولتاژ در حالت عادی کار می‌کند و با LPDS=1 در حالت کم‌مصرف قرار می‌گیرد. در حالت کم‌مصرف می‌توان با پیکربندی RAMLV=1 در رجیستر PWR_CTLR، حالت ولتاژ پایین RAM را فعال کرد تا مصرف توان کاهش یابد.

در حالت STANDBY هنگام تغذیهٔ عادی، با پیکربندی R2KSTY=1 در رجیستر PWR_CTLR می‌توان تغذیهٔ ۲ کیلوبایت RAM را حفظ کرد و با R30KSTY=1 مانع از قطع تغذیهٔ ۳۰ کیلوبایت RAM شد؛ هنگام تغذیه با VBAT نیز با پیکربندی R2KVBAT=1 در رجیستر PWR_CTLR تغذیهٔ ۲ کیلوبایت RAM حفظ می‌شود و با R32K_VBATEN=1 تغذیهٔ ۳۰ کیلوبایت RAM نگه داشته می‌شود. بر همین اساس می‌توان با پیکربندی RAMLV=1 در رجیستر PWR_CTLR، حالت ولتاژ پایین RAM را فعال کرد تا مصرف توان کاهش یابد.

۲.۱.۴ اجرای کد از SRAM

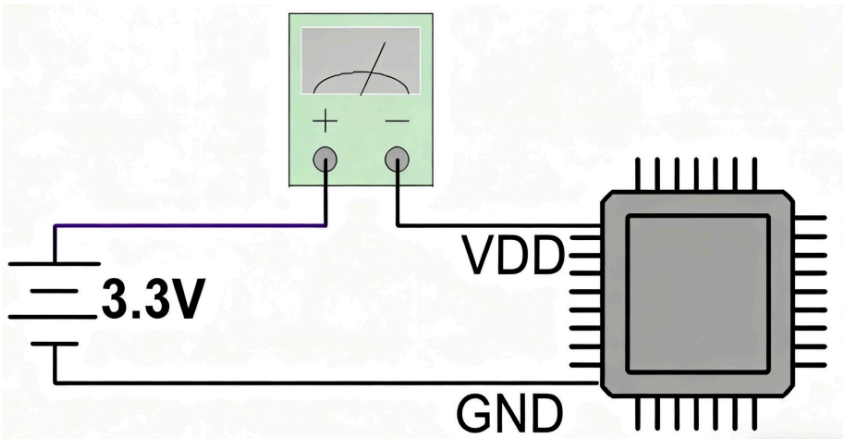
برای مثال در سری CH32L103، در حالت‌های SLEEP یا STOP می‌توان کد پردازش داده را در SRAM قرار داد تا اجرا شود، هم‌زمان Flash را در حالت کم‌مصرف پیکربندی کرد و با تنظیم LDOTRIM ولتاژ هستهٔ دیجیتال را کاهش داد تا مصرف توان پایین بیاید.

۲.۱.۵ بررسی مدارهای اطراف

- به جریان نشتی قطعات تبدیل توان خارجی توجه کنید: برخی درایورها و تراشه‌های تبدیل توان اگر با نرم‌افزار فعال شده باشند، پس از ورود به حالت کم‌مصرف بهتر است خاموش شوند، چون خودشان جریان کاری مصرف می‌کنند؛ اگر با سخت‌افزار فعال شده باشند، باید در دیتاشیت تراشه بررسی شود جریان کاری ساکن چقدر است و اگر نسبتاً بالا باشد، تنها راه، اصلاح مدار سخت‌افزاری یا جایگزینی با تراشهٔ کم‌مصرف است.
- آیا در مدار اطراف مسیر بستهٔ (حلقهٔ جریان) وجود دارد؟ برای مثال اگر مقاومت‌های تقسیم ولتاژ برداشت ولتاژ باز نشده باشند، با اینکه ADC خاموش شده است، این مدار تقسیم ولتاژ همچنان جریان مصرف می‌کند و میزان آن عموماً به مقاومت و ولتاژ تغذیه بستگی دارد؛ قطعات اطراف مانند سنسورها، ماژول‌های ارتباطی و LEDهای نشانگر باید پیش از ورود به حالت کم‌مصرف از تغذیه قطع شوند.
- آیا وضعیت GPIOها با هم تعارض دارد؟ بر اساس شماتیک دستگاه، مدار اتصال خارجی پایه‌های I/O را بررسی کنید. هنگامی‌که پایه از طریق مقاومت مدار اطراف به زمین کشیده شده (Pull-down خارجی) و پایه به‌صورت ورودی Pull-up پیکربندی شده باشد، مقاومت Pull-up داخلی تراشه و مقاومت Pull-down خارجی یک حلقه می‌سازند و جریان مصرف به این دو مقاومت بستگی دارد؛ به همین ترتیب وقتی پایه از طریق مقاومت مدار اطراف به تغذیه کشیده شده و پایه به‌صورت ورودی Pull-down پیکربندی شده باشد، مقاومت Pull-down داخلی و مقاومت Pull-up خارجی حلقهٔ جریان می‌سازند و جریان مصرف نیز به این دو مقاومت بستگی دارد.

۲.۱.۶ کاهش خطای اندازه‌گیری جریان

- هنگام اندازه‌گیری جریان مصرفی، تغذیهٔ ماژول‌های اطراف را قطع کنید (مانند سنسورها، تراشه‌های ارتباطی، LED و تراشهٔ مبدل USB به سریال) تا اثر نشتی قطعات خارجی حذف شود و فقط حداقل سیستم MCU باقی بماند.
- هنگام اندازه‌گیری واقعی جریان، اتصال اشکال‌زدای SWD را کاملاً قطع کنید و فقط خط تغذیه را باقی بگذارید؛ هنگام اندازه‌گیری جریان ساکن در حد میکروآمپر با مولتی‌متر، رنج باید به رنج µA منتقل شود تا دقت به‌خاطر انتخاب برد بزرگ‌تر از دست نرود.



شکل ۲-۱ نمودار اندازه‌گیری جریان

۲.۲ نکات مربوط به خواب و بیدارسازی

۲.۲.۱ خواب و بیدارسازی صحیح

- پیش از خواب، ساعت جانبی PWR را فعال کنید. در حالت‌های کم‌مصرف مختلف، منابع بیدارسازی متفاوت است؛ با مراجعه به دفترچهٔ کاربرد مطمئن شوید که حالت کم‌مصرف و منبع بیدارسازی متناظر و منطبق‌اند.
- از تحریف مداوم منبع بیدارسازی بپرهیزید؛ برای مثال وقفهٔ پیوسته مانع خواب می‌شود و پرچم وقفه باید به‌طور کامل پاک شود.
- پس از بیدارسازی، ساعت دوباره ساخته شود: پس از خروج از حالت توقف، HSI به‌طور خودکار ساعت سیستم می‌شود؛ اگر قبلاً از HSE/PLL استفاده می‌کرده‌اید باید دستی سوئیچ کنید (بدون بازپیکربندی ساعت، جانبی‌ها از کار می‌افتند) و پس از بیدارسازی تابع پیکربندی ساعت فراخوانی شود.
- بررسی کنید که حالت کم‌مصرف، شیوهٔ ورود به خواب (WFI/WFE) و شیوهٔ بیدارسازی (رویداد/وقفه) با هم منطبق باشند، خطوط EXTI به‌درستی نگاشت شده باشند و لبهٔ تحریف (بالارونده/پایین‌رونده) با سیگنال واقعی یکسان باشد؛ سیگنال باید الزامات لبه را برآورده کند.

۲.۲.۲ روش حفظ سطح منطقی پایه‌ها در حالت STANDBY

در حالت عادی هنگام ورود تراشه به حالت STANDBY، پایه‌ها وضعیت قبلی خود را حفظ نمی‌کنند و I/O به‌صورت پیش‌فرض ورودی آنالوگ می‌شود. اگر می‌خواهید تراشه وارد حالت STANDBY شود و سطح منطقی پایه‌ها حفظ گردد، این کار با روش زیر ممکن است:

۱ پایه را به‌صورت ورودی Pull-up/Pull-down پیکربندی کنید.

۲ ساعت AFIO را باز کنید.

۳ پایه‌ای که باید سطح آن حفظ شود به‌عنوان پایهٔ ورودی وقفهٔ خارجی پیکربندی و رجیستر AFIO_EXTICR[x] تنظیم می‌شود (برای مثال PA3: `AFIO->EXTICR[1]=0`).

۴ درخواست رویداد کانال وقفهٔ خارجی فعال و رجیستر EXT_EVENR تنظیم می‌شود (برای مثال PA3: `EXTI->EVENR|=(1<<3)`).

> **توجه:** فقط حفظ سطح در حالت ورودی Pull-up/Pull-down پشتیبانی می‌شود و حفظ در حالت خروجی پشتیبانی نمی‌شود. چون این قابلیت از طریق AFIO_EXTICR[x] عمل می‌کند و برای مثال PA3 و PB3 بیت یکسانی از رجیستر AFIO_EXTICR[x] را تنظیم می‌کنند، نمی‌توان سطح پایه‌های کانال EXTI یکسان را هم‌زمان حفظ کرد.

۲.۲.۳ روش دریافت منبع رویداد بیدارسازی WFE

بیدارسازی به شیوهٔ WFE نیازی به دخالت کنترلر وقفه ندارد. اگر از شیوهٔ WFE برای بیدارسازی دستگاه استفاده می‌کنید و نیاز به دریافت منبع بیدارسازی WFE دارید، این کار با روش زیر ممکن است:

۱ خط EXTI خارجی یا داخلی را در حالت رویداد پیکربندی کنید.

۲ سیگنال درخواست وقفهٔ خط EXTI مربوطه را فعال کنید (EXTI_INTENR).

۳ دستور `WFE` را اجرا کنید؛ پس از بیدارسازی، رجیستر پرچم وقفه (EXTI_INTFR) را بررسی و منبع رویداد بیدارسازی را تشخیص دهید.

◆ ۲.۲.۴ روش حفظ داده‌های RAM در حالت STANDBY

در حالت عادی در حالت STANDBY، ریگولاتور ولتاژ خاموش است و همهٔ مدارها به‌جز مدار بیدارسازی و مدارهای دامنهٔ پشتیبان از برق خارج می‌شوند؛ به‌طور پیش‌فرض همهٔ داده‌های RAM از دست می‌رود و پس از خروج در شرایط بیدارسازی مشخص‌شده، میکروکنترلر reset می‌شود.

برای مثال در CH32F/V20x_V30x در حالت آماده‌به‌کار، هنگام تغذیهٔ عادی با پیکربندی R2KSTY=1 در رجیستر PWR_CTLR می‌توان تغذیهٔ ۲ کیلوبایت RAM را حفظ کرد و با R30KSTY=1 تغذیهٔ ۳۰ کیلوبایت RAM را نگه داشت؛ هنگام تغذیه با VBAT با پیکربندی R2KVBAT=1 در رجیستر PWR_CTLR تغذیهٔ ۲ کیلوبایت RAM حفظ می‌شود و با R32K_VBATEN=1 تغذیهٔ ۳۰ کیلوبایت RAM نگه داشته می‌شود. بر همین اساس می‌توان با پیکربندی RAMLV=1 در رجیستر PWR_CTLR، حالت ولتاژ پایین RAM را فعال کرد تا مصرف توان کاهش یابد.

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول ۲ تاریخچهٔ نسخه‌های سند اصلی AN01000

تاریخ	نسخه	شرح تغییرات
2026/07/08	V1.0	انتشار نسخهٔ اول

◆ بیانیه

این سند کاربردی صرفاً جهت مرجع ارائه شده است و ترجمهٔ فارسی آن توسط تیم معمار الکترونیک صنعت تهیه شده است. نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول ۳ واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
SLEEP / STOP / STANDBY	سه حالت کم‌مصرف اصلی؛ از کم‌عمق‌ترین (SLEEP) تا کم‌مصرف‌ترین (STANDBY)
WFI	Wait For Interrupt؛ دستور ورود به خواب با بیدارسازی از طریق وقفه
WFE	Wait For Event؛ دستور ورود به خواب با بیدارسازی از طریق رویداد
RTC	Real-Time Clock؛ ساعت واقعی؛ بیدارسازی زمان‌بندی‌شده از STOP یا STANDBY را فراهم می‌کند
WKUP	پایهٔ اختصاصی بیدارسازی (PA0)؛ با لبهٔ بالارونده، MCU را از STANDBY بیدار می‌کند
AWU	Auto Wake-Up؛ ماژول بیدارسازی خودکار مبتنی بر شمارندهٔ ۶ بیتی، در MCUهای فاقد RTC
LPTIM	Low-Power Timer؛ تایمر کم‌مصرف ۱۶ بیتی در سری CH32L103 با منابع ساعت متنوع
PVD	Programmable Voltage Detector؛ پایشگر ولتاژ قابل‌برنامه‌ریزی برای نظارت بر VDD
HSE / HSI	اسیلاتور خارجی/داخلی فرکانس‌بالا (High/High-Speed Internal)
LSE / LSI	اسیلاتور خارجی/داخلی کم‌فرکانس؛ LSE معمولاً کریستال 32.768kHz است
PLL	حلقهٔ قفل فاز برای ضرب فرکانس و ساخت ساعت سیستم
PWR_CTLR	رجیستر کنترل توان؛ بیت‌های LPDS، RAMLV، R2KSTY، R30KSTY، R2KVBAT، R32K_VBATEN و PLS[2:0] در آن تعریف می‌شوند
SLEEPDEEP / PDDS	بیت‌های انتخاب عمق خواب و انتخاب حالت STANDBY هنگام ورود به کم‌مصرف
EXTI	کنترلر وقفه/رویداد خارجی؛ خط ۱۷ برای RTC، خط ۱۶ برای PVD و کانال ۲۱ برای LPTIM
PWR / BKP / AFIO	ساعت جانبی‌های مدیریت توان، دامنهٔ پشتیبان و پیکربندی جایگزین پایه‌ها
SRAM / Flash	حافظهٔ دادهٔ سریع / حافظهٔ برنامه
SWD	رابط اشکال‌زدایی سریال دو سیمه؛ برای اندازه‌گیری دقیق جریان باید کاملاً قطع شود
VBAT	تغذیهٔ باتری پشتیبان برای حفظ داده‌های RAM و دامنهٔ پشتیبان



معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN01000 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۱۸

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com